

The agent infrastructure problem

You built an agent, optimized its prompts, engineered its context, implemented an eval harness, and it works beautifully in development...

... then the infrastructure got in the way

-  Tools making API calls or querying databases are rate-limited.
-  Parallelized subagents/tool calls leads to resource contention and degraded performance.
-  Containers run out of memory, are killed by the scheduler, are preempted by other jobs.
-  Agent memory loss or corruption, wiping out precious context.
-  Agent components (MCPs, coder subagents) are easy to configure insecurely.

Agents Fail at Multiple Layers of the Orchestration Stack

The problem isn't that agents fail.

It's that recovering from failure is challenging without the full context of how the infrastructure, networking, logical, and semantic layers interact so that the agent can figure out how to recover from it.

What Agents are saying...



Agents can recover from infra-level failures, but only if we give them the right context.

Design Principles for Self-Healing Agents

1. **Plain Python/TS/JS** — DSLs incur additional cognitive overhead (for both humans and agents)
2. **Durability & observability hooks** — `@env.task`, `@flyte.trace` works with any framework/stack
3. **Make failures cheap** — global caching, run-level replay log, and state persistence
4. **Infrastructure as context** — agents see and fix OOM/network errors; request more resources
5. **Agent self-healing utilities** — secure tool building and orchestration
6. **Human-in-the-loop** — debugger and manual feedback when self-service isn't enough

Chapter 1: Where Agents Break

 Layer	❌ Example failures
Infrastructure	OOM, container killed or preempted, image pull backoff
Network	API rate-limiting, network timeouts, ephemeral service outage
Logical	Programming bugs, data validation errors
Semantic	Hallucinations, incorrect tool calls, context utilization errors
Tool execution	Bad arguments, tool timeouts, tool errors
Memory	State wiped or corrupted on crash, context erasure

The context and evaluation gap

Your evals typically test **semantic correctness**: *"Does it answer correctly?"*. Likewise, agents typically focus on recovering from semantic failures: *"I need more context about topic X to answer correctly"*.

Agents often don't directly reason about:

- Surviving a network timeout
- Recovering from system-level errors
- Recalling state across retries

💡 **Insight:** agents can recover from errors at all the layers of the agent stack, even infrastructure-level failures, but only with the right context and tools ✅.

Context engineering is also an infra problem

If a failure **wipes the agent's state**, that hard-earned context is worthless.

Durability: what even is it?

It's about making the following very easy:

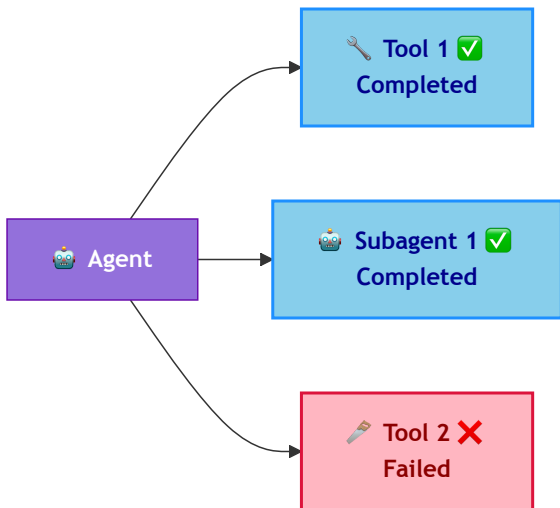
-  **Run-level replay log**: immediately resume an agent roll-out where it left off
-  **Global caching**: don't re-execute shareable compute/io-bound workloads
-  **Intermediate state persistence**: save and restore intermediate state between tasks

Replay log

A service that records the state of an agent and its subtasks (tools, subagents, etc.) at each step.

Avoids task re-execution, prevents memory/context loss, and even recovers from crashes in the root agent process.

▶ Normal Execution



📋 Replay Log

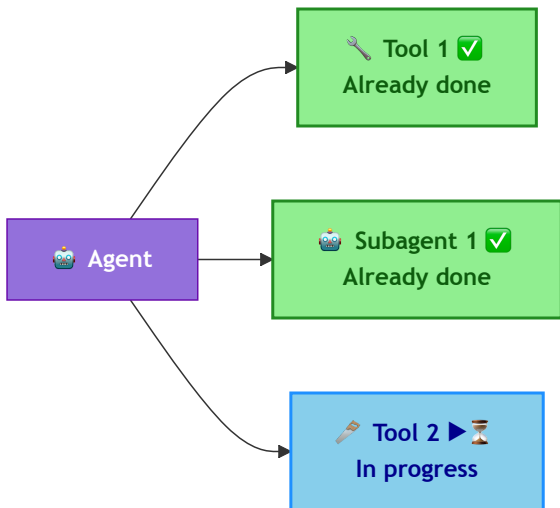
Step	Component	Status
1	🔧 Tool 1	✅
2	🤖 Subagent 1	✅
3	🔧 Tool 2	❌

Replay log

A service that records the state of an agent and its subtasks (tools, subagents, etc.) at each step.

Avoids task re-execution, prevents memory/context loss, and even recovers from crashes in the root agent process.

💣🔄 Crash Recovery



📋 Replay Log

Step	Component	Status
1	🔧 Tool 1	✅
2	🤖 Subagent 1	✅
3	🔧 Tool 2	🕒

Global Caching

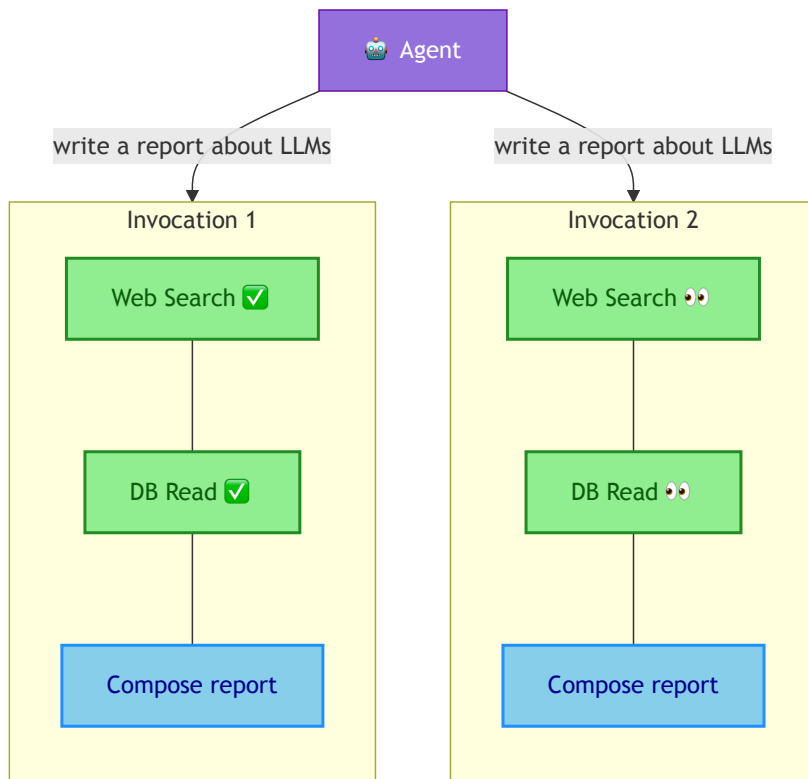
Avoid re-executing workloads where the output can be **shared across all agent runs**.

Control which tools/subagents are cached and which are not.

```
@env.task(cache="auto")
async def web_search_tool(url: str) -> str:
    ...

@env.task(cache="auto")
async def db_read_tool(query: str) -> str:
    ...

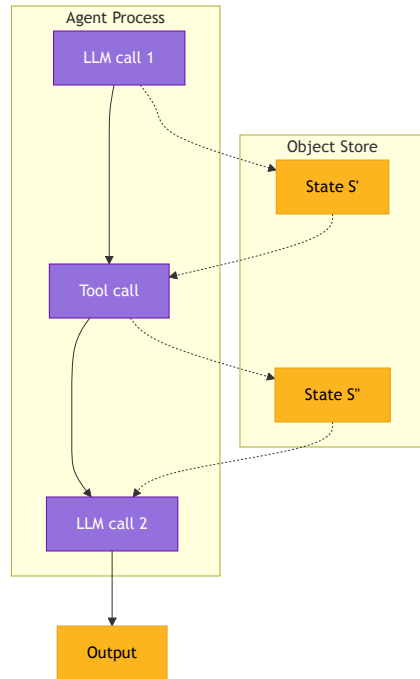
@env.task(cache="disable")
async def composer_subagent(content: str) -> None:
    ...
```



Intermediate state persistence

Abstract away serializing and deserializing state (memory, data) to object store.

```
class AgentState(BaseModel):  
    messages: list[Message]  
  
@env.task  
async def llm(state: AgentState) -> AgentState: ...  
  
@env.task  
async def tool(state: AgentState) -> AgentState: ...  
  
@env.task  
async def agent() -> str:  
    state = AgentState()  
    while not await done(state):  
        message = await llm(state)  
        if is_tool_call(state):  
            message = await tool(state)  
        state.messages.append(message)  
    return await compose_final_answer(state)
```



Chapter 2: Six Design Principles for Self-Healing Agents

1. **Plain Python/TS/JS** — DSLs incur additional cognitive overhead (for both humans and agents)
2. **Durability & observability hooks** — `@flyte.trace` , `@env.task` ; works with any framework/stack
3. **Make failures cheap** — global caching, run-level replay log, and state persistence
4. **Infrastructure as context** — agents see and fix OOM/network; request more resources
5. **Agent self-service utilities** — secure tool building and orchestration
6. **Human-in-the-loop** — debugger and manual feedback when self-service isn't enough

Plain Python/TS/JS

Or, any other general purpose programming language really.

🔄 Loops, fan-out, conditionals, and *try/except* are trivial. No DSL surprises.

★ Exceptions are a perfect delivery mechanism for critical context about failures at all layers of the stack.

🔗 Provide **functional hooks** to trace and checkpoint intermediate state — then get out of the AI engineer's way.

```
import flyte

env = flyte.TaskEnvironment("agent", image=flyte.Image.from_debian_base())

@flyte.trace
async def tool(x: int, y: int) -> AgentState: ...

@env.task(retries=RetryStrategy(5))
async def agent() -> str:
    state = AgentState()
    while not await done(state):
        message = await llm(state)
        if is_tool_call(state):
            try:
                message = await tool(parse_tool_call(state))
            except ToolArgsError as exc:
                message = await llm(
                    state, prompt=f"Fix the tool args: {str(exc)}"
                )
        state.messages.append(message)
    return await compose_final_answer(state)
```

Durability & observability hooks

Make it really easy to trace, checkpoint, and persist intermediate state.

`TaskEnvironment` :
infrastructure-level
configuration: container
image, resources, etc.

`@env.task` : containerized
functions that auto-persist
intermediate state

`@flyte.trace` : helper
functions inside a task that
are tracked by the replay log

```
import flyte
from flyte.io import File, DataFrame

image = flyte.Image.from_debian_base().with_pip_packages(["pandas"])
agent_env = flyte.TaskEnvironment("agent", image=image)
tool_env = flyte.TaskEnvironment(
    "tools", image=image, resources=flyte.Resources(cpu=4, memory="4Gi")
)

@flyte.trace
async def write_code(prompt: str) -> str: ...

@tool_env.task
async def run_code(code: str, data: DataFrame) -> File: ...

@agent_env.task
async def mle_agent(data: DataFrame) -> File:
    code = await write_code("Write a Python script to train a model on the data")
    return await run_code(code, data)
```

Make failures cheap

Failures are inevitable, but they don't have to be expensive.

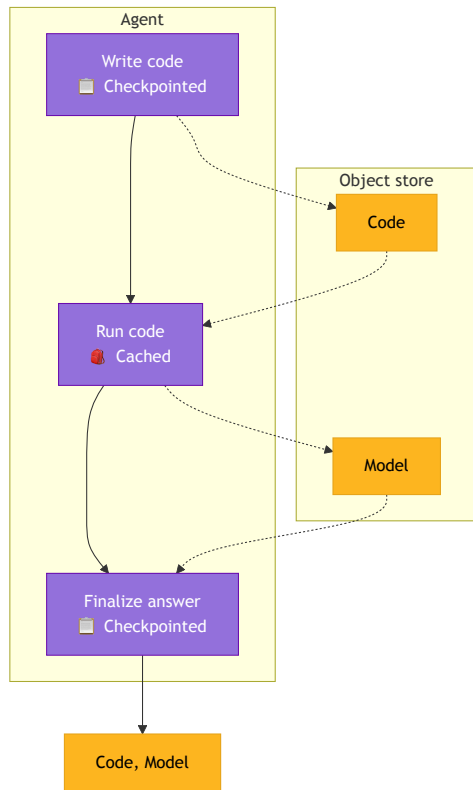
```
@flyte.trace
async def write_code(prompt: str) -> str: ...

@tool_env.task(cache="auto")
async def run_code(code: str, data: DataFrame) -> File: ...

@flyte.trace
async def finalize_answer(code: str, model: File) -> str: ...

@agent_env.task(retries=3)
async def mle_agent(prompt: str, data: DataFrame) -> tuple[str, File]:
    code = await write_code("Write a Python script to train a model...")
    model = await run_code(code, data)
    return await finalize_answer(code, model)
```

★ Failed runs become *training data* or *additional context* for the agent to learn from.



Infrastructure as context

Infra-level context relating to errors can be delivered via exception handling.

The Flyte SDK exposes system-level errors like `flyte.errors.OOMError` as exceptions.

Agents can catch and respond to system-/infra-level errors and adjust resource requests accordingly.

Adjustments are sent back to the platform imperatively until `max_iter` budget is exhausted.

```
@agent_env.task(retries=3)
async def mle_agent(data: DataFrame, max_iter: int) -> tuple[str, File]:
    code = await write_code("Write a Python script to train a model...")
    resources = {"memory": "128Mi", "cpu": 4}
    for _ in range(max_iter):
        try:
            resources = flyte.Resources(**resources)
            model = await run_code.override(resources=resources)(
                code, data
            )
            break
        except flyte.errors.OOMError as exc:
            resources = await adjust_resources(
                prompt=f"Encountered error {exc}, adjust the memory"
            )
    ...
```

Agent self-healing utilities: code sandbox

Multi-level context (from infra to semantic) + sandboxes = self-healing agents

Code sandbox runtime: agents build their own tools safely in an isolated, secure container.

MLE agent can write their own tools to train a model given some data.

Sandbox runs the code securely in a container.

In the case of an error, the agent re-writes the code with unit tests and tries again.

```
from flyte.extras import Sandbox

sandbox = Sandbox(packages=["pandas", "numpy"])

@agent_env.task(retries=3)
async def mle_agent(data: DataFrame, max_iter: int) -> File:
    code = await write_code("Write a Python script to train a model...")
    for _ in range(max_iter):
        try:
            return await sandbox.run.aio(
                code=code,
                inputs={"data": DataFrame},
                outputs={"model": File},
                data=data,
            )
        except flyte.errors.RuntimeUserError as exc:
            code, tests = await write_code_with_tests(
                f"Re-write code with tests\n{code}\nbased on error: {exc}."
            )
            await sandbox.run_tests.aio(code, tests)
```

Agent self-healing utilities: orchestration sandbox

Multi-level context (from infra to semantic) + sandboxes = self-healing agents

Agents write pipeline code to orchestrate trusted tools.

Orchestration "code-mode" sandbox runs agent-generated pipeline, which dispatches tool calls to the external tasks.

Tight error-iteration loop to fix orchestration code bugs.

```
@tool_env.task
async def process_data(data: DataFrame) -> File: ...

@tool_env.task
async def train_model(data: DataFrame) -> File: ...

...

@agent_env(retries=3)
async def mle_agent(data: DataFrame, max_iter: int) -> File:
    tools = [process_data, train_model, ...]
    code = await write_pipeline_code("Write a model training pipeline...", tools)
    pipeline = flyte.sandboxed.code_to_task(code, functions=tools)
    for _ in range(max_iter):
        try:
            best_model = await pipeline(data)
        except flyte.errors.RuntimeUserError as exc:
            code = await write_pipeline_code(
                f"Re-write code\n{code}\nbased on error: {exc}."
            )
            pipeline = flyte.sandboxed.code_to_task(code, functions=tools)
```

Human-in-the-loop recourse

Human-in-the-loop gates: when agents can't recover by themselves, this is the ultimate fallback for course correction or providing missing context.

When `max_iter` is exhausted, get more context from a human (or external system)

Recursively call the agent with the additional context.

```
import flyteplugins.hitl as hitl

agent_env = flyte.TaskEnvironment(..., depends_on=[hitl.env])

@agent_env(retries=3)
async def mle_agent(
    data: DataFrame, max_iter: int, ctx: str | None = None
) -> File:
    ...
    for _ in range(max_iter):
        try:
            return await pipeline(data)
        except flyte.errors.RuntimeUserError as exc:
            # code re-write handling
            ...

    event = await hitl.new_event.aio(
        "get_more_context",
        data_type=str,
        prompt="Model training failed, please provide more context.",
    )
    more_context = await event.wait.aio()
    mle_agent(data, max_iter, ctx=more_context)
```

Chapter 3: What This Looks Like in Practice

Dragonfly case study — How Dragonfly scales agentic research across 250k products

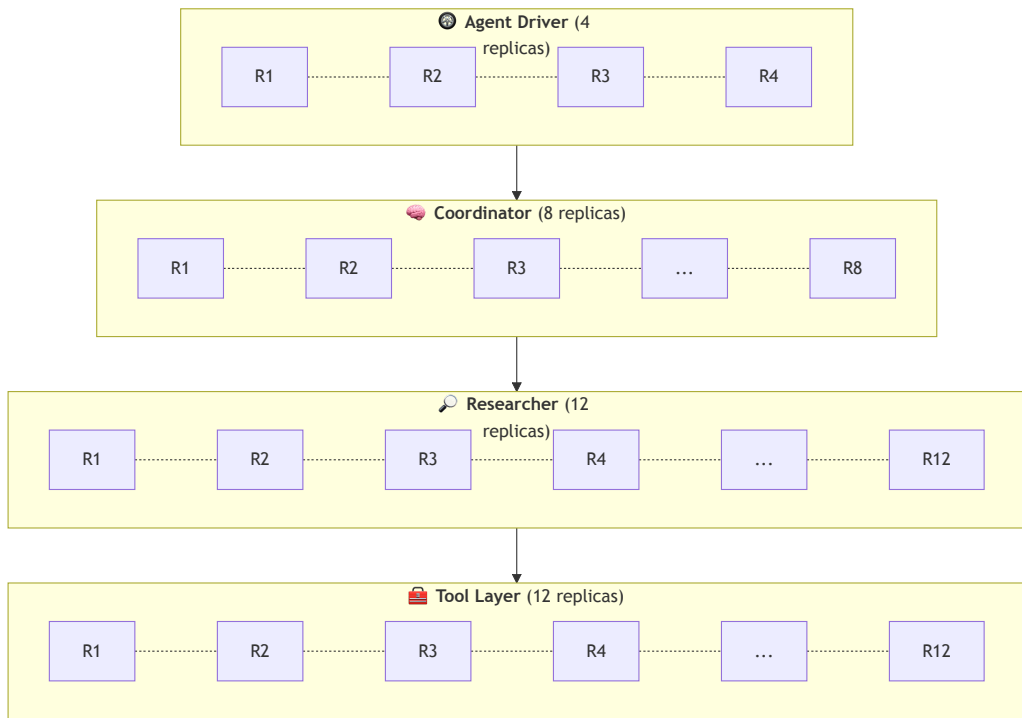
🤖 **Challenge:** Build an automated solutions architect – an agent that creates and a living knowledge graph of SaaS products.

- 250K+ software products.
- ~200 steps per agent call
- ~100 LLM calls per product

Scaling deep research agents

✓ **Solution:** Tiered task environments on Flyte 2.

- **Cross-run caching:** pay for LLM API calls once.
- **Semantic convergence detection:** coordinator consolidates overlapping research streams.
- **Checkpoint-based recovery:** spot instance interruptions become a non-issue.
- **Full auditability:** every agent decision and tool call is traced.



Scaling deep research agents

 **Results:** 1 hour from local prototype to production-grade remote workflows.

- 2,000+ concurrent research runs
- 50%  failure recovery time
- 30%  development velocity
- 12 hours/week saved on infrastructure.

Conclusion

Tomorrow, do your agents a favor:

- **Observability is necessary but not sufficient:** a durability layer helps agents recover quickly.
- **Don't aim for failure-proof:** aim for **cheap failures**, **fast recovery**, and **eval feedback**
- **Try/except is critical:** errors are a natural delivery mechanism to give the agent critical context.

Help your agents help themselves.

Thank You

Learn more: [Union.ai](#) — come talk at the booth.

Questions?